



I. PORTADA

Tema:	Búsqueda en una rejilla
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	7mo A
Alumnos participantes:	Ojeda Castillo Jonathan Fabricio Christian Sánchez
Asignatura:	Inteligencia Artificial
Docente:	Ing. Rubén Nogales, Mg.

1. Objetivos

1.1 Objetivo general

Implementar y analizar diferentes algoritmos de búsqueda no informada para encontrar un camino entre un estado inicial y un estado objetivo dentro de una rejilla, comparando su funcionamiento, eficiencia y capacidad para obtener soluciones óptimas.

1.2 Objetivos específicos

- Implementar los algoritmos de búsqueda en anchura (BFS), búsqueda en profundidad (DFS), búsqueda de profundidad iterativa (IDDFS) y búsqueda bidireccional sobre una rejilla de 10×20 .
- Analizar el comportamiento de cada algoritmo mediante el número de nodos expandidos, la longitud del camino encontrado y el tiempo de ejecución.
- Comparar los resultados obtenidos para determinar qué algoritmo presenta un mejor desempeño en función de la optimalidad y eficiencia de la búsqueda.

2. Modalidad

No presencial.

3. Tiempo de duración

- **Presenciales:** 0 horas.
- **No presenciales:** 3 horas.



4. Instrucciones

Para el desarrollo de la práctica se implementó un problema de búsqueda sobre una rejilla de 10×20 , estableciendo una posición inicial y una posición objetivo.

Desde cada casilla se permiten cuatro movimientos:

- arriba,
- abajo,
- izquierda,
- derecha.

Estos movimientos solamente son válidos cuando la nueva posición permanece dentro de los límites de la rejilla.

Sobre el mismo problema se aplicaron cuatro algoritmos de búsqueda no informada:

- **BFS (Breadth-First Search):** utiliza una cola FIFO y explora los estados nivel por nivel.
- **DFS (Depth-First Search):** utiliza una pila LIFO y explora una rama hasta alcanzar la mayor profundidad posible antes de retroceder.
- **IDDFS (Iterative Deepening Depth-First Search):** ejecuta búsquedas DFS limitadas en profundidad, incrementando progresivamente el límite.
- **Búsqueda bidireccional:** realiza simultáneamente búsquedas desde el estado inicial y desde el estado objetivo hasta encontrar un punto de encuentro.

Cada algoritmo construye un árbol de búsqueda mediante objetos **Nodo**, que mantienen referencias hacia su nodo padre y hacia sus nodos hijos. Posteriormente, se reconstruye el camino solución y se comparan los resultados de los diferentes algoritmos.

5. Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computador personal.
- Python.
- Editor o entorno de desarrollo para programación.
- Archivos fuente del proyecto (.py).
- Sistema de archivos para almacenar los resultados de la ejecución.



TAC (Tecnologías para el Aprendizaje y Conocimiento)

- ☐ Plataformas educativas.
- ☐ Simuladores y laboratorios virtuales.
- ☒ Aplicaciones educativas.
- ☒ Recursos audiovisuales.
- ☐ Gamificación.
- ☒ Inteligencia Artificial.

Otros: Lenguaje de programación Python y herramientas de desarrollo de software.

6. Actividades por desarrollar

Para el desarrollo de la práctica se realizaron las siguientes actividades:

1. Modelar una rejilla de 10×20 como un problema de búsqueda.
2. Definir los estados, acciones posibles, estado inicial y estado objetivo.
3. Implementar la clase **Nodo** para representar los elementos del árbol de búsqueda.
4. Implementar las estructuras de datos necesarias: pila para las búsquedas en profundidad y cola para las búsquedas en anchura.
5. Implementar el algoritmo BFS.
6. Implementar el algoritmo DFS.
7. Implementar el algoritmo IDDFS mediante búsquedas DFS limitadas.
8. Implementar el algoritmo de búsqueda bidireccional.
9. Reconstruir el camino desde el nodo solución hasta el estado inicial.
10. Representar gráficamente el camino encontrado sobre la rejilla.
11. Generar y visualizar los árboles de búsqueda.
12. Medir el tiempo de ejecución y contabilizar los nodos expandidos.
13. Comparar los resultados obtenidos por los cuatro algoritmos.
14. Determinar el algoritmo o algoritmos que encontraron el camino más corto.



7. Resultados obtenidos

Para iniciar el programa se debe ejecutar el archivo `main.py`, que constituye el punto de entrada de la aplicación. Desde la terminal, ubicándose en el directorio principal del proyecto, se puede ejecutar mediante:

```
python main.py
```

Al iniciar el programa se muestra una interfaz que permite seleccionar la forma en que se determinarán el estado inicial y el estado objetivo del problema.

La rejilla utilizada por el programa tiene 10 filas y 20 columnas. Considerando índices que comienzan en cero, las filas pueden tomar valores entre 0 y 9, mientras que las columnas pueden tomar valores entre 0 y 19.

El programa ofrece tres opciones para establecer los estados inicial y objetivo:

1. **Introducción manual:** permite introducir las coordenadas del estado inicial y del estado objetivo. Las coordenadas deben corresponder a posiciones válidas dentro de la rejilla de 10×20 .
2. **Generación aleatoria:** genera automáticamente las posiciones inicial y objetivo de forma aleatoria dentro de la rejilla.
3. **Configuración predeterminada:** utiliza directamente las posiciones establecidas por defecto en el programa.

Para los resultados presentados en esta sección se utilizó la configuración predeterminada:

$$\text{Inicio} = (0, 0), \quad \text{Objetivo} = (9, 19).$$

Es decir, la búsqueda comienza en la casilla ubicada en la esquina superior izquierda de la rejilla y termina en la esquina inferior derecha.

7.1 Descripción del problema

El programa modela la rejilla de 10×20 casillas como un problema de búsqueda. Cada casilla representa un estado y desde una casilla se puede realizar un movimiento en cuatro direcciones:

$$\{\text{ARRIBA, ABAJO, IZQUIERDA, DERECHA}\}.$$

Un movimiento es válido únicamente cuando la nueva posición permanece dentro de los límites de la rejilla.

De esta manera, el problema de búsqueda queda definido por un estado inicial, un estado objetivo, un conjunto de acciones posibles y una función de transición que determina el estado resultante de aplicar cada acción.

Sobre este mismo problema se ejecutan cuatro algoritmos de búsqueda no informada.



Cuadro 1: Algoritmos implementados

Algoritmo	Archivo	Estructura	Estrategia
BFS	bfs.py	Cola (FIFO)	Explora nivel por nivel
DFS	dfs.py	Pila (LIFO)	Explora una rama en profundidad
IDDFS	iddfs.py	Pila + límite creciente	Repite DFS limitado
Bidireccional	bidireccional.py	Dos colas	Busca desde ambos extremos

Cada algoritmo, al finalizar, proporciona el nodo solución o `None` en caso de no encontrar una solución, el número de nodos expandidos y la raíz del árbol de búsqueda generado. En el caso de la búsqueda bidireccional se conservan dos raíces: una correspondiente al estado inicial y otra correspondiente al estado objetivo.

El archivo `main.py` se encarga de ejecutar los cuatro algoritmos, medir sus tiempos de ejecución, obtener las soluciones, dibujar los caminos encontrados sobre la rejilla, comparar los resultados y almacenar la información obtenida en `resultados.txt`.

7.2 El árbol de búsqueda

El árbol de búsqueda está formado por objetos `Nodo` enlazados entre sí. Cada objeto `Nodo` contiene referencias hacia su padre y hacia sus hijos.

La clase puede representarse conceptualmente mediante:

```
class Nodo:
    def __init__(self, estado, padre=None,
                  accion=None, profundidad=0):

        self.estado = estado
        self.padre = padre
        self.accion = accion
        self.hijos = []
        self.profundidad = profundidad

    def agregar_hijo(self, hijo):
        self.hijos.append(hijo)
```

El atributo `padre` constituye el enlace ascendente del árbol y permite reconstruir el camino desde un nodo solución hasta la raíz.

El atributo `hijos` constituye el enlace descendente y permite recorrer posteriormente el árbol para visualizarlo.

Por ejemplo, un fragmento del árbol puede representarse como:

```
raíz (estado=(0,0), padre=None)
hijo A (estado=(0,1), padre=raíz, acción="DERECHA")
```



```
nieto (estado=(0,2), padre=hijo A, acción="DERECHA")
```

```
...
```

```
hijo B (estado=(1,0), padre=raíz, acción="ABAJO")
```

```
...
```

La raíz siempre es el nodo creado a partir de `problema.inicial`, excepto en el árbol correspondiente al objetivo de la búsqueda bidireccional, donde la raíz representa `problema.objetivo`.

Cada vez que un algoritmo genera un sucesor, este se crea como un nuevo objeto `Nodo`, se conecta con su padre mediante `agregar_hijo()` y posteriormente se incorpora a la estructura que administra la frontera.

7.3 Atributo de profundidad

Existe un detalle de implementación relacionado con el atributo `profundidad` de la clase `Nodo`.

Al crear un hijo se utiliza:

```
Nodo(nuevo_estado, nodo, accion)
```

por lo que no se proporciona explícitamente la profundidad del nuevo nodo. En consecuencia, el valor permanece en 0.

La forma correcta de actualizar este atributo sería:

```
Nodo(  
    nuevo_estado,  
    nodo,  
    accion,  
    nodo.profundidad + 1  
)
```

Este detalle no afecta los resultados obtenidos por BFS, DFS ni por la búsqueda bidireccional. Tampoco afecta el funcionamiento de IDDFS, debido a que este algoritmo controla la profundidad mediante una variable independiente.

En IDDFS se utiliza una estructura equivalente a:

(nodo, profundidad)

Por tanto, la profundidad utilizada por IDDFS no depende de `nodo.profundidad`.

7.4 Pila y cola

Los archivos `pila.py` y `cola.py` implementan estructuras sencillas sobre listas de Python.

La pila utiliza:

```
apilar = append  
desapilar = pop()
```



por lo que sigue una estrategia LIFO (*Last In, First Out*).
La cola utiliza:

```
encolar = append  
desencolar = pop(0)
```

por lo que sigue una estrategia FIFO (*First In, First Out*).

La estructura utilizada para administrar la frontera determina el orden en que los nodos son explorados:

- BFS utiliza una cola.
- DFS utiliza una pila.
- IDDFS utiliza una pila con un límite de profundidad.
- La búsqueda bidireccional utiliza dos colas.

El tipo de objeto utilizado para representar los nodos, sin embargo, es el mismo en todos los algoritmos.

7.5 Problema de la rejilla

El archivo `problema.py` define el problema independientemente del algoritmo de búsqueda.

Sus principales operaciones son:

- `acciones(estado)`: devuelve los movimientos válidos desde una determinada casilla.
- `resultado(estado, accion)`: aplica una acción y devuelve el nuevo estado.
- `es_objetivo(estado)`: comprueba si el estado coincide con el objetivo.

De esta manera, los algoritmos de búsqueda no necesitan conocer directamente los detalles de la rejilla.

Todos interactúan con el problema mediante una interfaz común. Esto permite reutilizar los algoritmos para otros problemas si se implementa una nueva clase con las mismas operaciones.

7.6 Crecimiento del árbol en los algoritmos

El patrón general de expansión utilizado por BFS y DFS es:

1. Extraer un nodo de la estructura correspondiente.
2. Comprobar si el nodo representa el estado objetivo.



3. Si es el objetivo, devolver la solución.
4. Si no es el objetivo, generar sus sucesores.
5. Comprobar cuáles de los sucesores todavía no han sido visitados.
6. Crear un objeto **Nodo** para cada sucesor válido.
7. Añadir el nuevo nodo como hijo del nodo actual.
8. Insertar el nuevo nodo en la estructura que administra la frontera.

La principal diferencia entre BFS y DFS se encuentra en el orden en que los nodos pendientes son extraídos de la frontera.

7.7 BFS

BFS (*Breadth-First Search*) utiliza una cola FIFO.

Como el primer nodo que entra en la cola es el primero que sale, los nodos son explorados nivel por nivel.

Primero se generan los hijos de la raíz, después los nietos, posteriormente los nodos del siguiente nivel, y así sucesivamente.

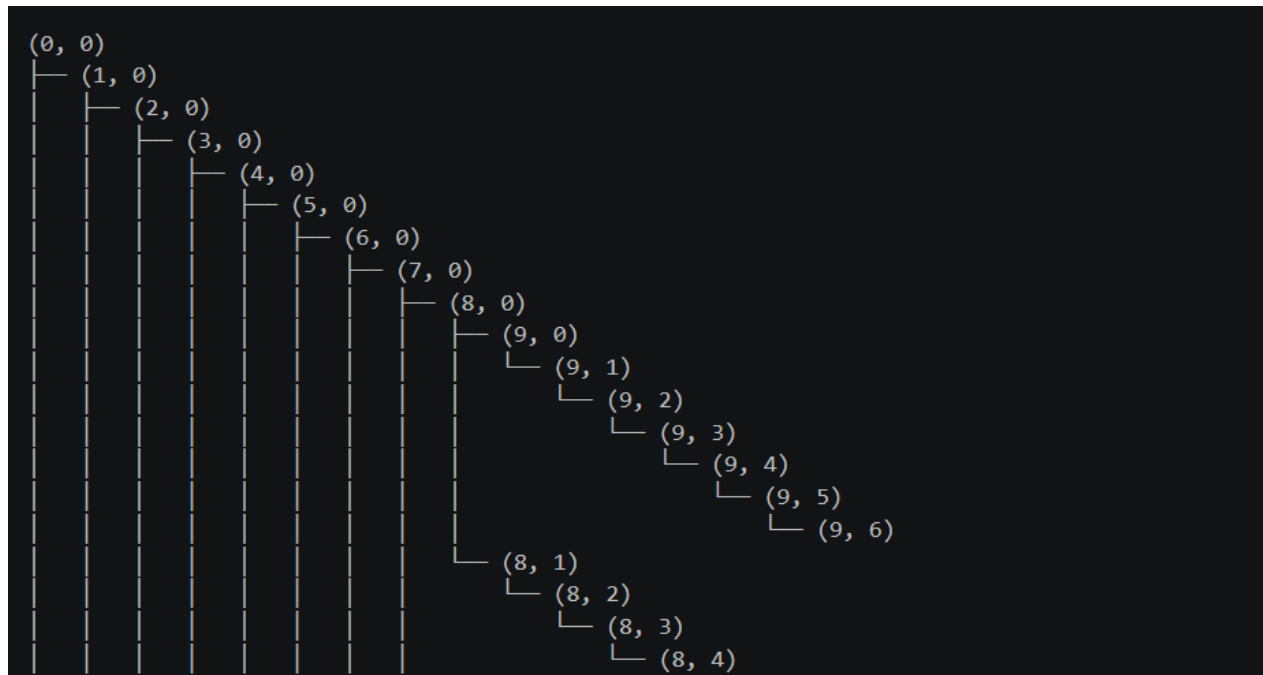


Figura 1: Parte del árbol de búsqueda generado por BFS.

En este problema todas las acciones tienen el mismo costo, por lo que BFS garantiza encontrar un camino de longitud mínima.



Para la configuración utilizada,

$$(0, 0) \longrightarrow (9, 19),$$

BFS encontró el camino:

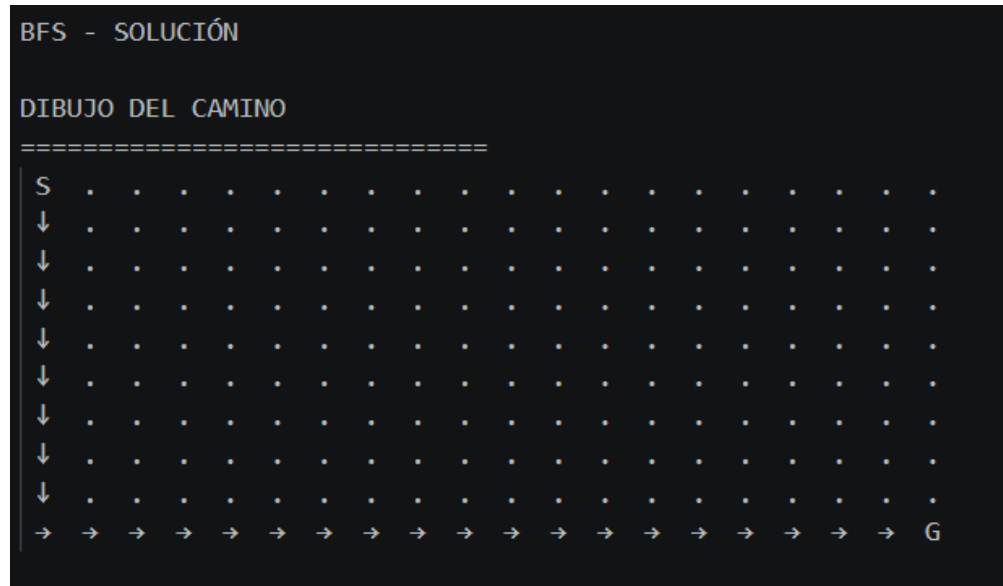


Figura 2: Camino encontrado por BFS.

$$(0, 0) \rightarrow (1, 0) \rightarrow \dots \rightarrow (9, 0) \\ \rightarrow (9, 1) \rightarrow \dots \rightarrow (9, 19).$$

La solución tiene longitud:

$$L_{\text{BFS}} = 28.$$

Este valor es óptimo, ya que se necesitan al menos 9 movimientos verticales y 19 movimientos horizontales para alcanzar $(9, 19)$ desde $(0, 0)$:

$$d = 9 + 19 = 28.$$

Durante la ejecución se expandieron:

$$199 \text{ nodos.}$$

El tiempo de ejecución medido fue:

$$0,0003688 \text{ s.}$$



7.8 DFS

DFS (*Depth-First Search*) utiliza una pila LIFO.

La búsqueda se introduce en una rama hasta que no encuentra nuevos sucesores y posteriormente continúa con otras ramas pendientes.

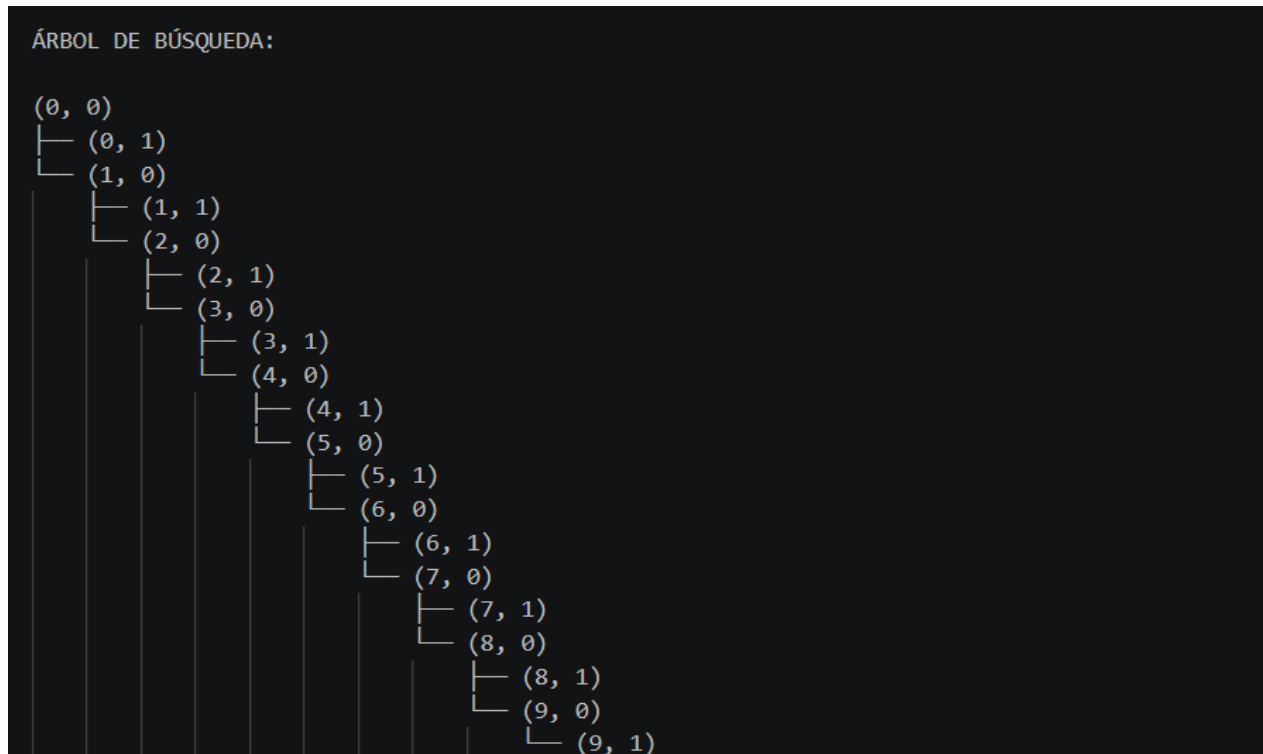


Figura 3: Parte del árbol de búsqueda generado por DFS.

En la ejecución utilizada, DFS encontró una solución

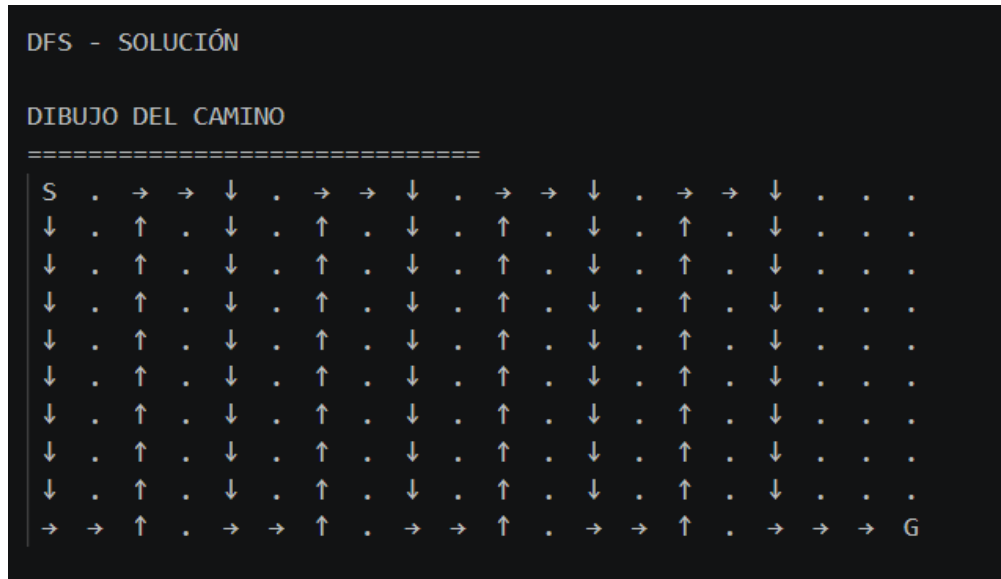


Figura 4: Camino encontrado por DFS.

de longitud:

$$L_{DFS} = 100.$$

Esta solución es considerablemente más larga que la solución óptima de longitud 28. Esto confirma que DFS no garantiza encontrar el camino más corto.

El camino encontrado presenta numerosos movimientos hacia arriba y hacia abajo antes de alcanzar finalmente el objetivo. Por tanto, aunque el algoritmo encontró una solución sin necesidad de explorar toda la rejilla, la solución obtenida no fue óptima.

El número de nodos expandidos fue:

$$118.$$

El tiempo de ejecución fue:

$$0,0002376 \text{ s.}$$

En este caso, DFS fue ligeramente más rápido que BFS y expandió menos nodos, pero produjo una solución mucho peor en términos de longitud.

La complejidad temporal considerada para DFS es:

$$O(3^m),$$

mientras que su complejidad espacial es:

$$O(3^m),$$

donde m representa la profundidad máxima alcanzada por el árbol de búsqueda.



7.9 IDDFS

IDDFS (*Iterative Deepening Depth-First Search*) combina las características de DFS y BFS mediante la realización de sucesivas búsquedas DFS limitadas por profundidad.

El límite comienza en cero y se incrementa progresivamente:

$$0, 1, 2, 3, \dots$$

hasta encontrar una solución.

El procedimiento puede representarse de la siguiente manera:

```
def iddfs(problema):  
  
    nodos_expandidos = 0  
    limite = 0  
  
    while True:  
  
        solucion, expandidos, raiz = \  
            dfs_limitado(problema, limite)  
  
        nodos_expandidos += expandidos  
  
        if solucion is not None:  
            return solucion, nodos_expandidos, raiz  
  
        limite += 1
```

Cada llamada a `dfs_limitado()` construye un nuevo árbol desde cero.

Los árboles correspondientes a los intentos fallidos dejan de estar referenciados después de terminar cada iteración. El árbol que finalmente se conserva es el correspondiente a la búsqueda que encuentra la solución.

El número de nodos expandidos sí se acumula entre todas las iteraciones, por lo que representa el trabajo total realizado por IDDFS.

Para la configuración $(0, 0) \rightarrow (9, 19)$, IDDFS encontró una solución

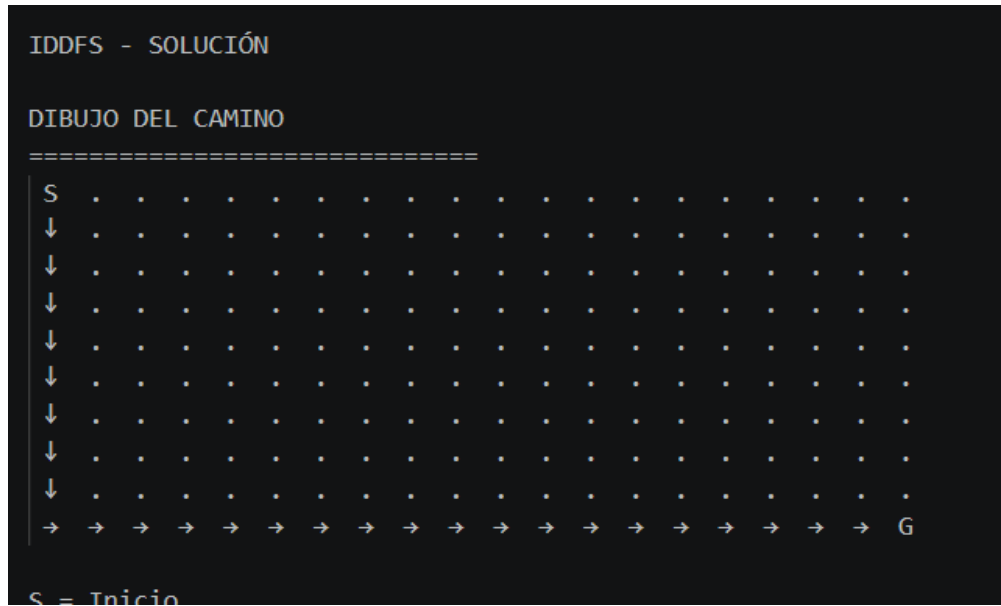


Figura 5: Camino encontrado por IDDFS.

de longitud:

$$L_{IDDFS} = 28.$$

Por tanto, obtuvo una solución óptima, al igual que BFS.

Sin embargo, el costo computacional fue considerablemente mayor. IDDFS expandió:

9426 nodos,

y tuvo un tiempo de ejecución de:

$$0,0208146 \text{ s.}$$

Esto se debe a que los niveles anteriores se vuelven a explorar en cada nueva iteración del límite de profundidad.

La complejidad temporal considerada es:

$$O(3^d),$$

y la complejidad espacial:

$$O(3^d),$$

donde d representa la profundidad de la solución.



7.10 Búsqueda bidireccional

La búsqueda bidireccional mantiene dos búsquedas simultáneas:

- una desde el estado inicial;
- otra desde el estado objetivo.

Por tanto, existen dos árboles:

- `raiz_inicial`, que representa el árbol que crece desde el inicio;
- `raiz_objetivo`, que representa el árbol que crece desde el objetivo.

Cada lado mantiene su propia cola y sus estructuras de estados visitados.

El proceso alterna las expansiones de ambos lados. Cuando un estado generado por una búsqueda ya se encuentra en el conjunto de estados visitados por la búsqueda contraria, se ha encontrado un punto de encuentro.

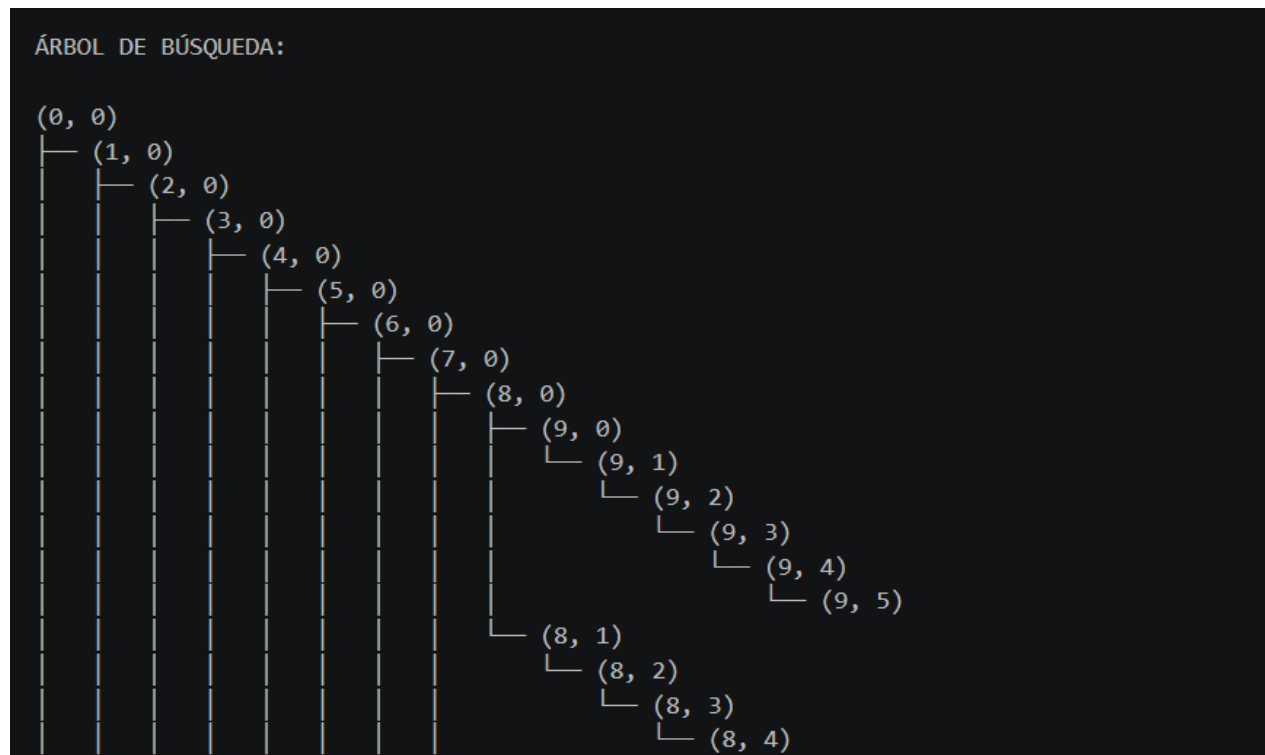


Figura 6: Parte del árbol de búsqueda generados por la búsqueda bidireccional desde el inicio.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS,
ELECTRÓNICA E INDUSTRIAL
CICLO ACADÉMICO: FEBRERO–JULIO 2026



$$\begin{aligned}(0, 0) &\rightarrow (1, 0) \rightarrow (2, 0) \rightarrow (3, 0) \rightarrow (4, 0) \\ &\rightarrow (4, 1) \rightarrow \dots \rightarrow (4, 19) \\ &\rightarrow (5, 19) \rightarrow (6, 19) \rightarrow (7, 19) \rightarrow (8, 19) \rightarrow (9, 19).\end{aligned}$$

El número de nodos expandidos fue:

181.

El tiempo de ejecución fue:

0,0005135 s.

La ventaja teórica de la búsqueda bidireccional consiste en que, para un problema adecuado, puede reducir el crecimiento de la búsqueda aproximadamente de

$$O(3^d)$$

a

$$O(3^{d/2}),$$

donde d es la profundidad de la solución.

En este caso, la complejidad temporal y espacial considerada es:

$$O(3^{d/2}).$$

Reconstrucción de la solución bidireccional

Cuando ambas búsquedas se encuentran, existen dos nodos que representan la misma casilla, pero pertenecen a árboles diferentes.

Para construir un único camino desde el estado inicial hasta el estado objetivo, se recorre el árbol generado desde el objetivo hacia su raíz.

Como este árbol fue construido en la dirección

objetivo $\longrightarrow$ encuentro,

es necesario invertir las acciones para obtener la dirección

encuentro $\longrightarrow$ objetivo.

Las acciones opuestas utilizadas son:

ARRIBA $\leftrightarrow$ ABAJO, IZQUIERDA $\leftrightarrow$ DERECHA.

Por esta razón se utiliza el diccionario de acciones opuestas:

_ACCIONES_OPUESTAS



Durante la reconstrucción pueden crearse nuevos nodos que son agregados al árbol inicial mediante `agregar_hijo()`.

Por esta razón, una parte de la rama observada en la representación final del árbol puede corresponder al proceso de reconstrucción y no exclusivamente a la exploración original.

7.11 Comparación de resultados

Los resultados obtenidos para la configuración

$$(0, 0) \rightarrow (9, 19)$$

se resumen en la siguiente tabla.

Cuadro 2: Resultados obtenidos por los algoritmos

Algoritmo	Longitud	Nodos expandidos	Tiempo (s)
BFS	28	199	0.0003688
DFS	100	118	0.0002376
IDDFS	28	9426	0.0208146
Bidireccional	28	181	0.0005135

La distancia mínima entre $(0, 0)$ y $(9, 19)$ es:

$$|9 - 0| + |19 - 0| = 9 + 19 = 28.$$

Por lo tanto, BFS, IDDFS y la búsqueda bidireccional encontraron soluciones óptimas de longitud 28.

DFS, en cambio, encontró una solución de longitud 100, por lo que no fue óptima.

En cuanto al número de nodos expandidos, DFS obtuvo el menor valor con 118 nodos, seguido por la búsqueda bidireccional con 181 y BFS con 199. IDDFS presentó un número considerablemente mayor de expansiones, con 9426, debido a la repetición de las búsquedas con diferentes límites de profundidad.

En tiempo de ejecución, DFS fue el más rápido en esta ejecución concreta, seguido por BFS, la búsqueda bidireccional e IDDFS. Estos tiempos corresponden exclusivamente a la ejecución realizada y pueden variar en otras ejecuciones, especialmente cuando las posiciones inicial y objetivo se generan aleatoriamente.

7.12 Comparación de los algoritmos

Para el análisis teórico se utilizan las siguientes variables:



Cuadro 3: Variables utilizadas en el análisis de complejidad

Variable	Significado
b	Factor de ramificación efectivo
d	Profundidad de la solución óptima
m	Profundidad máxima alcanzada por la búsqueda

En la rejilla utilizada, cada casilla puede tener como máximo cuatro movimientos posibles. Sin embargo, cuando se evita regresar inmediatamente al estado anterior, el número de sucesores nuevos puede ser como máximo tres. Por esta razón, para el análisis de complejidad se considera un factor de ramificación efectivo de

$$b = 3.$$

Este valor representa el número máximo de nuevos estados que pueden generarse a partir de un estado ya alcanzado, excluyendo el movimiento que conduce directamente al estado anterior.

Cuadro 4: Comparación teórica de los algoritmos de búsqueda

Algoritmo	Tiempo	Espacio	Óptimo	Característica
BFS	$O(b^d) = O(3^d)$	$O(b^d) = O(3^d)$	Sí	Busca por niveles
DFS	$O(b^m) = O(3^m)$	$O(bm) = O(3m)$	No	Explora una rama hasta el fondo
IDDFS	$O(b^d) = O(3^d)$	$O(bd) = O(3d)$	Sí	DFS con límites crecientes
Bidireccional	$O(b^{d/2}) = O(3^{d/2})$	$O(b^{d/2}) = O(3^{d/2})$	Sí*	Busca desde ambos extremos

* La optimalidad de la búsqueda bidireccional supone que las búsquedas se realizan por niveles sobre un espacio de estados no ponderado, como ocurre en la rejilla utilizada en esta práctica.

7.12.1 Interpretación de la notación O

La notación O describe el crecimiento asintótico de los recursos requeridos por un algoritmo a medida que aumenta el tamaño del problema.

En primer lugar, las complejidades pueden expresarse de forma general en función del factor de ramificación b .

Por ejemplo, para BFS se tiene:

$$O(b^d) = O(3^d).$$



Esto indica que, al considerar $b = 3$, el crecimiento es exponencial respecto a la profundidad d .

De manera similar, para DFS:

$$O(b^m) = O(3^m),$$

representa un crecimiento exponencial respecto a la profundidad máxima m .

Para el espacio utilizado por DFS se tiene:

$$O(bm) = O(3m).$$

Como $b = 3$ es una constante, esta expresión presenta un crecimiento lineal respecto a la profundidad m .

En el caso de IDDFS, la complejidad temporal es:

$$O(b^d) = O(3^d),$$

mientras que la complejidad espacial es:

$$O(bd) = O(3d).$$

La búsqueda bidireccional puede reducir considerablemente el número de estados explorados porque, en condiciones adecuadas, cada búsqueda necesita recorrer aproximadamente la mitad de la profundidad total.

Su complejidad puede expresarse como:

$$O(b^{d/2}) = O(3^{d/2}).$$

Por ejemplo, al comparar BFS con la búsqueda bidireccional:

$$O(b^d) = O(3^d)$$

frente a

$$O(b^{d/2}) = O(3^{d/2}),$$

se observa que la segunda expresión puede ser considerablemente menor cuando d es grande.

7.12.2 Optimalidad y características

Para comparar los algoritmos no solo debe considerarse si encuentran una solución óptima, sino también si garantizan encontrar una solución cuando esta existe y qué cantidad de tiempo y memoria requieren.



BFS

BFS es un algoritmo completo y óptimo cuando todas las acciones tienen el mismo costo. En la rejilla utilizada en esta práctica, cada movimiento tiene costo unitario. Por lo tanto, BFS garantiza encontrar un camino con el menor número posible de movimientos.

Su principal desventaja es el consumo de memoria, ya que debe mantener en la frontera un gran número de nodos correspondientes a los niveles que todavía no han sido explorados.

Por tanto, BFS ofrece una solución óptima, pero puede requerir una cantidad considerable de memoria.

DFS

DFS es completo únicamente bajo determinadas condiciones, por ejemplo, cuando el espacio de búsqueda es finito y se controlan adecuadamente los estados repetidos.

DFS no es óptimo, ya que puede encontrar una solución antes que otra de menor longitud.

Su principal ventaja es el menor consumo de memoria en comparación con BFS, debido a que mantiene principalmente la rama que está explorando y las alternativas pendientes.

Por tanto, DFS puede utilizar menos memoria y encontrar rápidamente una solución, pero no garantiza que dicha solución sea óptima.

IDDFS

IDDFS es completo y óptimo cuando todas las acciones tienen el mismo costo.

El algoritmo realiza sucesivas búsquedas en profundidad con límites crecientes. Por esta razón, la primera solución encontrada se encuentra en la menor profundidad posible.

Su principal ventaja es que combina la optimalidad de BFS con un consumo de memoria considerablemente menor, similar al de una búsqueda en profundidad.

La principal desventaja es que debe volver a explorar los niveles anteriores en cada nueva iteración, lo que aumenta el tiempo total de ejecución.

Por tanto, IDDFS obtiene una solución óptima y utiliza poca memoria, pero puede realizar un mayor número de expansiones debido a la repetición de las búsquedas.

Búsqueda bidireccional

La búsqueda bidireccional puede ser completa y óptima cuando se aplica sobre un espacio de estados no ponderado y ambas búsquedas se realizan por niveles.

En la práctica desarrollada, todos los movimientos tienen costo unitario y las dos búsquedas utilizan colas. Por esta razón, la búsqueda bidireccional puede encontrar un camino de longitud mínima.

Su principal ventaja es que reduce la profundidad que debe recorrer cada búsqueda. En condiciones adecuadas, esto permite reducir tanto el tiempo como el número de nodos almacenados respecto a una búsqueda unidireccional por niveles.

Sin embargo, requiere mantener dos fronteras y detectar correctamente el punto de encuentro entre ambas búsquedas.



Por tanto, en este problema la búsqueda bidireccional obtiene una solución óptima y puede ser más eficiente que BFS en problemas con una profundidad de solución grande.

8. Conclusiones

La práctica permitió comprender e implementar diferentes estrategias de búsqueda no informada aplicadas a un problema representado mediante una rejilla.

Se comprobó que, aunque BFS, DFS, IDDFS y la búsqueda bidireccional trabajan sobre el mismo espacio de estados, la estructura utilizada para administrar la frontera determina significativamente el orden en que se exploran los nodos y, por tanto, el comportamiento del algoritmo.

BFS utiliza una cola FIFO y explora los estados por niveles. Debido a que todos los movimientos de la rejilla tienen el mismo costo, garantiza encontrar un camino de longitud mínima. En la ejecución realizada obtuvo una solución de 3 movimientos.

DFS utiliza una pila LIFO y explora una rama hasta alcanzar la mayor profundidad posible antes de retroceder. En el experimento encontró una solución de 5 movimientos. Esto demuestra que una menor cantidad de nodos expandidos no implica necesariamente una solución óptima.

IDDFS combina la estrategia de profundidad de DFS con la propiedad de optimalidad de BFS para problemas con costos uniformes. Para encontrar la solución realiza varias búsquedas con diferentes límites de profundidad, acumulando el trabajo realizado en cada iteración. En la ejecución analizada encontró una solución óptima de 3 movimientos.

La búsqueda bidireccional realiza búsquedas simultáneas desde el estado inicial y desde el estado objetivo. En esta ejecución obtuvo una solución óptima de 3 movimientos y expandió únicamente 4 nodos, por lo que presentó el mejor resultado experimental entre los algoritmos evaluados.

Finalmente, se comprobó que la representación mediante objetos **Nodo**, conectados mediante relaciones de padre e hijos, permite no solamente reconstruir los caminos solución, sino también representar y visualizar los árboles de búsqueda generados durante la ejecución de los algoritmos.

9. Recomendaciones

- Realizar pruebas con diferentes posiciones iniciales y objetivos para comprobar cómo varía el comportamiento de cada algoritmo.
- Ejecutar las búsquedas sobre rejillas de diferentes tamaños para analizar de mejor manera el crecimiento del espacio de estados.
- Registrar el tiempo de ejecución además del número de nodos expandidos para obtener una comparación más completa.



- Corregir la actualización del atributo profundidad de la clase Nodo, de manera que represente correctamente el nivel del nodo dentro del árbol.
- Mantener una representación independiente entre los nodos generados durante la exploración y los nodos incorporados durante la reconstrucción del camino en la búsqueda bidireccional.
- Evitar la duplicación del encabezado del árbol de búsqueda en la generación del archivo de resultados.
- Realizar múltiples ejecuciones con diferentes estados iniciales y objetivos para obtener resultados estadísticamente más representativos.
- Comparar no solamente la longitud del camino, sino también el tiempo de ejecución y el número de nodos expandidos.

10. Referencias bibliográficas

- [1] Universidad Técnica de Ambato, “Página institucional UTA,” Universidad Técnica de Ambato, 2025. [Online]. Available: <https://www.uta.edu.ec/v4.0/>. [Accessed: Aug. 20, 2026].
- [2] Universidad Técnica de Ambato, “Biblioteca virtual,” Universidad Técnica de Ambato, 2025. [Online]. Available: <https://www.uta.edu.ec/v4.0/index.php/biblioteca/biblioteca-virtual>. [Accessed: Aug. 20, 2026].
- [3] Universidad Técnica de Ambato, “Repositorio institucional,” Universidad Técnica de Ambato, 2025. [Online]. Available: <https://repositorio.uta.edu.ec/home>. [Accessed: Aug. 20, 2026].
- [4] Dirección de Educación Continua a Distancia y Virtual – DEaDV, “Sitio web institucional,” Universidad Técnica de Ambato, 2025. [Online]. Available: <https://deadv.uta.edu.ec/>. [Accessed: Aug. 20, 2026].
- [5] M. S. Ramírez Montoya, G. Zavala Enríquez, A. Martínez Arboleda, and M. Á. Montoya Bayardo, *Horizontes en el aprendizaje vinculado con el desarrollo sostenible: Nuevas vías en la era digital*. Octaedro, 2025.
- [6] J. Lázaro and R. De Medeiros, “Avances y discusiones sobre el uso de inteligencia artificial (IA) en la educación,” *Revista de Educación y Tecnología*, vol. 30, no. 1, pp. 29–45, 2024. [Online]. Available: http://www.scielo.org.pe/scielo.php?script=sci_arttext&pid=S1019-94032024000100029. [Accessed : Aug,20, 2026].
- [7] T. Moreno Olivos and A. Ramírez Elías, “Evaluación formativa y retroalimentación del aprendizaje,” in *Evaluación en entornos virtuales de aprendizaje*, pp. 67–89. UNAM,



2024. [Online]. Available: <https://cuaieed.unam.mx/publicaciones/libro-evaluacion/pdf/Capitulo-04-EVALUACION-FORMATIVA-Y-RETROALIMENTACION.pdf>. [Accessed: Aug. 20, 2026].

- [8] A. E. Copado Rodríguez, *Experiencias de evaluación formativa en el Doctorado en Educación Multimodal desde la etnografía digital*, Tesis de Doctorado, Universidad Autónoma de Tlaxcala, Centro de Investigación Educativa, 2024. [Online]. Available: <https://centrodeinvestigacioneducativauatx.org/tesis-doctorado-en-educacion-2024>. [Accessed: Aug. 20, 2026].

““

A. Anexos

A.1 Anexo A. Código fuente de los algoritmos

En este anexo se incluye el código fuente correspondiente a los algoritmos:

- BFS.
- DFS.
- IDDFS.
- Búsqueda bidireccional.

A.2 Anexo B. Clase Nodo y estructuras de datos

Se incluye el código correspondiente a:

- clase Nodo;
- clase Pila;
- clase Cola.

A.3 Anexo C. Capturas de la ejecución

Se presentan capturas de la ejecución del programa y de los resultados obtenidos.

A.4 Anexo D. Árboles de búsqueda

Se incluyen las representaciones de los árboles generados por los algoritmos BFS, DFS, IDDFS y búsqueda bidireccional.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS,
ELECTRÓNICA E INDUSTRIAL
CICLO ACADÉMICO: FEBRERO–JULIO 2026



A.5 Anexo E. Caminos solución

Se incluyen las representaciones gráficas de los caminos encontrados sobre la rejilla.

A.6 Anexo F. Archivo de resultados

Se incluye el archivo `resultados.txt` generado por el programa con los resultados de las búsquedas.